

Unified Customer Satisfaction Tracking

Luca Scherzer
Claranet GmbH
Mannheim, Germany
luca.scherzer@claranet.com

Lars Beer
Atos SE
Mannheim, Germany
lars.beer@atos.net

Lukas Herberling
Claranet GmbH
Mannheim, Germany
lukas.heberling@claranet.com

Abstract—Tracking customer satisfaction is a crucial part for online businesses. For producers, it enables them to incorporate feedback about their products from customers directly, while for marketplace owners, it enables them to identify highly sought-after and badly selling products, and adapt their supply chain accordingly. We introduce Customer Satisfaction System (CuSaSy), a software that can be integrated with a large variety of marketplace systems, enabling marketplace operators to outsource customer satisfaction management to an outside party, decreasing time-to-market and increasing speed of development.

Index Terms—E-Business, Customer Satisfaction

I. EINLEITUNG

Die digitale Transformation des Handels hat die Art und Weise, wie Unternehmen mit ihren Kunden interagieren, grundlegend verändert. Im traditionellen stationären Einzelhandel konnte die Kundenzufriedenheit durch direkte persönliche Interaktion und unmittelbares Feedback erfasst werden. Verkaufspersonal hatte die Möglichkeit, durch Gespräche, Beobachtung der Körpersprache und spontane Rückmeldungen ein Gefühl für die Zufriedenheit ihrer Kunden zu entwickeln. Im elektronischen Handel (E-Commerce) hingegen fehlt diese persönliche Komponente weitgehend. Die räumliche und zeitliche Entkopplung zwischen Anbieter und Kunde erfordert systematische, skalierbare Ansätze zur Erfassung und Auswertung der Kundenzufriedenheit [1].

Kundenzufriedenheit hat sich als kritischer Erfolgsfaktor für Online-Geschäfte etabliert und beeinflusst direkt die Kundenbindung, Mundpropaganda-Marketing und letztendlich die Rentabilität des Unternehmens [2]. In einer zunehmend wettbewerbsintensiven digitalen Landschaft, in der Kunden mit wenigen Klicks zu Konkurrenzangeboten wechseln können, wird die systematische Messung und Verbesserung der Kundenzufriedenheit zu einem entscheidenden Wettbewerbsvorteil. Studien zeigen, dass zufriedene Kunden nicht nur wiederkehren, sondern auch als Markenbotschafter fungieren und durch positive Bewertungen und Empfehlungen neue Kunden akquirieren [3].

A. Motivation und Problemstellung

Die Erfassung der Kundenzufriedenheit dient mehreren Stakeholdern im E-Commerce-Ökosystem und erfüllt unterschiedliche, aber komplementäre Funktionen. Für Produkthersteller und -produzenten ermöglicht die systematische Sammlung von Kundenfeedback datengetriebene Produktverbesserungen und Iterationen basierend auf tatsächlichen Kundenerfahrungen [4]. Durch die Analyse von Bewertungen, Rezensionen und Beschwerden kön-

nen Hersteller Schwachstellen in ihren Produkten identifizieren, Verbesserungspotenziale erkennen und ihre Entwicklungsressourcen gezielt einsetzen. Dies führt zu einem kontinuierlichen Verbesserungsprozess, der die Produktqualität erhöht und die Kundenzufriedenheit langfristig steigert.

Marktplatzbetreiber profitieren von der Identifikation sowohl gut als auch schlecht performender Produkte. Diese Erkenntnisse ermöglichen es ihnen, ihre Produktkataloge zu optimieren, Prioritäten in der Lieferkette anzupassen und die Gesamtqualität des Marktplatzes zu verbessern [5]. Produkte mit durchweg negativen Bewertungen können aus dem Sortiment genommen oder herabgestuft werden, während besonders beliebte Produkte durch verbesserte Verfügbarkeit und prominentere Platzierung gefördert werden können. Darüber hinaus ermöglicht die Integration von Kundenzufriedenheitsdaten in Such- und Empfehlungsalgorithmen eine verbesserte Produktauffindbarkeit (Product Discovery), wodurch ein positiver Kreislauf entsteht: Bessere Produkte gewinnen an Sichtbarkeit, was zu mehr Verkäufen führt, während minderwertige Produkte natürlich herausgefiltert werden.

Für alle Beteiligten bietet die Einbindung von Zufriedenheitsdaten in die Suchfunktionalität erhebliche Vorteile. Kunden finden schneller die Produkte, die ihren Anforderungen entsprechen, was die Conversion-Rate erhöht und Retouren reduziert. Die Qualität der Suchergebnisse wird durch die Berücksichtigung von Bewertungen und Ratings signifikant verbessert, was das gesamte Einkaufserlebnis optimiert [6].

Trotz dieser offensichtlichen Bedeutung stellt die Implementierung robuster Systeme zur Erfassung der Kundenzufriedenheit erhebliche Herausforderungen dar. Viele E-Commerce-Plattformen, insbesondere kleinere Marktplätze und spezialisierte Online-Shops, verfügen nicht über die Ressourcen oder das technische Know-how, um umfassende Zufriedenheitsmanagementsysteme intern zu entwickeln. Die Entwicklung solcher Systeme erfordert Expertise in verschiedenen Bereichen: Frontend-Entwicklung für Bewertungsformulare, Backend-Entwicklung für die Datenverwaltung, Datenbank-Design für die effiziente Speicherung und Abfrage von Feedback-Daten, sowie Analytics-Kompetenz für die Auswertung und Visualisierung der gesammelten Informationen.

Diese Ressourcenknappheit führt zu mehreren problematischen Konsequenzen: verzögerte Markteinführung (Time-to-Market) neuer Features, inkonsistente Praktiken bei der Datenerfassung, die die Vergleichbarkeit und Aussagekraft der Daten beeinträchtigen, und verpasste Gelegenheiten für gezieltes Kundenengagement [7]. Kleinere Anbieter sind

oft gezwungen, zwischen der schnellen Einführung ihrer Kernprodukte und der Implementierung eines ausgereiften Feedback-Systems zu wählen, wobei letzteres häufig vernachlässigt wird.

Ein weiteres Problem besteht in der Fragmentierung der Datenerfassung. Verschiedene Touchpoints im Customer Journey – Website, Mobile App, E-Mail-Kommunikation, Support-Tickets – generieren unterschiedliche Arten von Feedback, die oft in separaten Systemen gespeichert werden. Diese Fragmentierung erschwert eine holistische Sicht auf die Kundenzufriedenheit und verhindert die Identifikation systemischer Probleme, die über mehrere Kanäle hinweg auftreten.

B. Forschungsziele

Diese Arbeit stellt das **Customer Satisfaction System (CuSaSy)** vor, eine modulare Softwarelösung, die entwickelt wurde, um die beschriebenen Herausforderungen zu adressieren. CuSaSy ermöglicht es Marktplatzbetreibern, das Management der Kundenzufriedenheit an ein dediziertes, spezialisiertes System auszulagern, das mit verschiedenen Marktplatz-Plattformen integriert werden kann. Durch diesen Ansatz können sich Betreiber auf ihre Kernkompetenzen konzentrieren, während die komplexen Aspekte der Feedback-Erfassung und -Analyse von einem spezialisierten System übernommen werden.

Die primären Ziele dieser Arbeit sind:

- 1) **Entwurf und Implementierung eines einheitlichen Systems zur Erfassung der Kundenzufriedenheit**, das mit verschiedenen E-Commerce-Plattformen integriert werden kann. Das System soll plattformunabhängig sein und standardisierte Schnittstellen bieten, die eine einfache Integration ermöglichen, unabhängig von der verwendeten Shop-Software.
- 2) **Bereitstellung einer skalierbaren Architektur**, die die Erfassung, Verarbeitung und Analyse von Kundenfeedback effizient handhabt. Das System muss in der Lage sein, sowohl kleine Shops mit wenigen hundert Transaktionen pro Monat als auch große Marktplätze mit Millionen von Interaktionen zu unterstützen, ohne dass die Performance leidet.
- 3) **Ermöglichung von Echtzeit-Einblicken** durch interaktive Dashboards und Analytics-Schnittstellen. Betreiber sollen in der Lage sein, Trends zu erkennen, Probleme frühzeitig zu identifizieren und datengestützte Entscheidungen zu treffen. Die Visualisierungen müssen sowohl aggregierte Übersichten als auch detaillierte Drill-Down-Analysen unterstützen.
- 4) **Gewährleistung von Datensicherheit und Datenschutzkonformität** bei der Verarbeitung sensibler Kundendaten. Das System muss DSGVO-konform sein und Best Practices für Datensicherheit implementieren, einschließlich Verschlüsselung, Zugriffskontrolle und Audit-Logging.
- 5) **Reduktion der Time-to-Market** für E-Commerce-Plattformen, die Zufriedenheitstracking implementieren möchten. Durch die Bereitstellung eines ferti-

gen Systems mit klaren Integrationspunkten können neue Marktplätze innerhalb von Tagen statt Monaten einsatzbereit sein.

C. Beitrag und Umfang

Unser Beitrag ist sowohl praktischer als auch technischer Natur. Wir präsentieren eine vollständige Implementierung von CuSaSy, die folgende Komponenten umfasst:

Backend-Architektur: Ein RESTful API-Backend, entwickelt mit Node.js und Express, das die Erfassung und Verwaltung von Feedback-Daten ermöglicht. Das Backend implementiert alle notwendigen Endpunkte für CRUD-Operationen (Create, Read, Update, Delete) auf Bewertungen, Ratings und Kommentaren. Es bietet eine saubere, gut dokumentierte API-Schnittstelle, die von verschiedenen Client-Anwendungen genutzt werden kann.

Frontend-Lösung: Eine moderne, webbasierte Benutzeroberfläche, entwickelt mit Next.js, React und TypeScript, für die Visualisierung und Analyse der gesammelten Daten. Das Frontend bietet interaktive Dashboards mit verschiedenen Visualisierungstypen (Liniendiagramme, Balkendiagramme, Fortschrittsanzeigen), eine detaillierte Tabellenansicht für die Analyse einzelner Bewertungen, und responsive Design für die Nutzung auf verschiedenen Endgeräten. Die Verwendung von Tremor UI-Komponenten gewährleistet eine konsistente, professionelle Benutzeroberfläche.

Integrationsfähigkeiten: Definierte Schnittstellen für die Integration mit externen E-Commerce-Systemen. Das System kann über Webhooks Events empfangen (z.B. "Bestellung abgeschlossen", "Produkt ausgeliefert") und basierend darauf Feedback-Anfragen auslösen. Es bietet auch Export-Funktionen für die Integration der Daten in bestehende CRM- oder Analytics-Systeme.

Authentifizierung und Autorisierung: Mechanismen für Multi-Tenant-Szenarien, die es mehreren Marktplätzen ermöglichen, das gleiche CuSaSy-System zu nutzen, während ihre Daten strikt voneinander getrennt bleiben. Das System implementiert rollenbasierte Zugriffskontrolle (RBAC) mit verschiedenen Benutzerrollen (z.B. Administrator, Analyst, Read-Only-Benutzer).

Datenexport-Funktionalität: Schnittstellen für die Integration mit Data Warehouses und Business Intelligence Tools. Das System unterstützt den Export von Daten in verschiedenen Formaten (CSV, JSON, Parquet) und bietet sowohl Batch-Export als auch kontinuierliche Synchronisation über Change Data Capture (CDC) Mechanismen.

Die Systemarchitektur folgt Microservice-Prinzipien, wodurch eine unabhängige Skalierung und Bereitstellung der Komponenten ermöglicht wird. Frontend und Backend können separat deployt und skaliert werden, was eine flexible Anpassung an unterschiedliche Lastszenarien ermöglicht. Durch die Bereitstellung als eigenständiger Service reduzieren wir die Komplexität für Marktplatzbetreiber erheblich, während gleichzeitig Flexibilität und Anpassungsmöglichkeiten erhalten bleiben. Betreiber können das System entweder als Software-as-a-Service (SaaS) nutzen oder in ihrer eigenen Infrastruktur deployen (On-Premise oder Private Cloud).

Die Implementierung demonstriert, wie moderne Web-Technologien und Best Practices aus dem Software Engineering genutzt werden können, um ein robustes, wartbares und erweiterbares System zu schaffen. Der Fokus liegt auf Klarheit und Verständlichkeit des Codes, um zukünftige Erweiterungen und Anpassungen zu erleichtern – ein Prinzip, das auch in der begleitenden AGENTS.md-Dokumentation betont wird.

D. Struktur der Arbeit

Der Rest dieser Arbeit ist wie folgt strukturiert:

Kapitel II bietet theoretischen Hintergrund zu E-Kundenzufriedenheit und E-Business-Prinzipien. Es werden etablierte Modelle und Frameworks zur Messung von Kundenzufriedenheit im digitalen Kontext vorgestellt, darunter das Expectation-Confirmation Model, das DeLone & McLean IS Success Model, und spezifische E-Service-Quality-Frameworks wie E-S-QUAL.

Kapitel III untersucht verwandte Arbeiten und existierende Lösungen im Bereich Customer Satisfaction Tracking. Es werden kommerzielle Angebote, Open-Source-Lösungen und akademische Prototypen analysiert und in einer Vergleichsmatrix gegenübergestellt, um die Einordnung von CuSaSy zu ermöglichen.

Kapitel IV präsentiert das Systemdesign und die Architektur von CuSaSy. Es werden die Gesamtarchitektur, Sicherheitsüberlegungen einschließlich Authentifizierung und Autorisierung, die Integration mit Data Warehouses, sowie die Datenerfassungs- und Verarbeitungspipeline detailliert beschrieben.

Kapitel V bietet eine ausführliche technische Dokumentation der Implementierung. Es umfasst die API-Spezifikation mit allen verfügbaren Endpunkten, Datenspeicherung und -verwaltung, Frontend-Architektur und Komponentenstruktur, sowie Deployment- und Konfigurationsaspekte. Dieses Kapitel ermöglicht es Lesern, die das System nie gesehen haben, ein vollständiges Verständnis der Implementierung zu erlangen.

Kapitel VI evaluiert die Effektivität des Systems und diskutiert Limitationen. Es werden Performance-Metriken, Usability-Aspekte, und potenzielle Verbesserungen behandelt.

Kapitel VII schließt mit einer Zusammenfassung der Erkenntnisse und Richtungen für zukünftige Arbeiten ab. Es werden mögliche Erweiterungen wie maschinelles Lernen für Sentiment-Analyse, erweiterte Integrationsmöglichkeiten, und zusätzliche Analytics-Features diskutiert.

II. THEORETISCHER HINTERGRUND

Dieses Kapitel legt die konzeptionellen und theoretischen Grundlagen für das Customer Satisfaction System (CuSaSy) dar. Es werden zunächst fundamentale Konzepte des E-Business und E-Commerce erläutert, gefolgt von etablierten Modellen zur Messung der Kundenzufriedenheit im digitalen Kontext. Anschließend werden technische Grundlagen wie Service-orientierte Architekturen und Zugriffskontrollmechanismen diskutiert, die für die Systemimplementierung relevant sind.

A. E-Business und E-Commerce Grundlagen

a) Definition und Abgrenzung:

E-Business bezeichnet die umfassende Integration digitaler Technologien in alle Geschäftsprozesse eines Unternehmens, während E-Commerce sich spezifisch auf die elektronische Abwicklung von Handelstransaktionen konzentriert [5]. E-Commerce umfasst den Kauf und Verkauf von Produkten oder Dienstleistungen über elektronische Netzwerke, primär das Internet.

Die wichtigsten E-Commerce-Geschäftsmodelle lassen sich wie folgt kategorisieren:

Business-to-Consumer (B2C): Unternehmen verkaufen direkt an Endkunden. Beispiele sind Online-Shops wie Amazon oder Zalando. Dies ist das für CuSaSy primäre Anwendungsgebiet, da hier die Kundenzufriedenheit unmittelbar messbar ist.

Business-to-Business (B2B): Transaktionen zwischen Unternehmen, etwa Großhandelsplattformen oder Beschaffungsportale. Auch hier spielt Kundenzufriedenheit eine Rolle, allerdings mit anderen Metriken als im B2C-Bereich.

Consumer-to-Consumer (C2C): Plattformen wie eBay oder Kleinanzeigen, wo Privatpersonen untereinander handeln. Hier ist beidseitiges Feedback (Käufer und Verkäufer) besonders relevant.

Marketplace-Modelle: Hybride Plattformen, die verschiedene Verkäufer aggregieren. Hier muss Zufriedenheit sowohl auf Produkt- als auch auf Plattformebene erfasst werden.

b) Erfolgsfaktoren im E-Commerce:

Das DeLone und McLean Information Systems Success Model [7] identifiziert sechs Dimensionen des Erfolgs von Informationssystemen, die sich direkt auf E-Commerce-Systeme übertragen lassen:

- 1) **System Quality:** Technische Aspekte wie Zuverlässigkeit, Verfügbarkeit und Antwortzeiten
- 2) **Information Quality:** Relevanz, Genauigkeit und Vollständigkeit der bereitgestellten Informationen
- 3) **Service Quality:** Qualität des Supports und der Kundenbetreuung
- 4) **Use/Intention to Use:** Tatsächliche oder beabsichtigte Systemnutzung
- 5) **User Satisfaction:** Zufriedenheit der Nutzer mit dem System
- 6) **Net Benefits:** Gesamtnutzen für alle Stakeholder

Für E-Commerce-Plattformen ist die User Satisfaction eine zentrale Dimension, die direkt die Wiedernutzung (Use) und damit den wirtschaftlichen Erfolg (Net Benefits) beeinflusst. CuSaSy adressiert primär die Erfassung und Verbesserung der User Satisfaction.

B. Kundenzufriedenheit im digitalen Kontext

a) Theoretische Grundlagen der Kundenzufriedenheit:

Kundenzufriedenheit wird klassisch definiert als das Ergebnis eines kognitiven Vergleichsprozesses zwischen erwarteter und tatsächlich erfahrener Leistung. Oliver's Expectation-Confirmation Theory [4] bildet die theoretische Grundlage für das Verständnis von Zufriedenheit:

Erwartungsbildung: Kunden entwickeln vor dem Kauf Erwartungen an ein Produkt oder eine Dienstleistung, basierend auf Marketingkommunikation, Bewertungen anderer Kunden und früheren Erfahrungen.

Wahrgenommene Leistung: Während und nach dem Kauf nehmen Kunden die tatsächliche Leistung wahr.

Konfirmation/Diskonfirmation: Kunden vergleichen die wahrgenommene Leistung mit ihren Erwartungen. Positive Diskonfirmation (Leistung übertrifft Erwartungen) führt zu hoher Zufriedenheit, negative Diskonfirmation zu Unzufriedenheit.

Zufriedenheitsurteil: Das Ergebnis dieses Vergleichsprozesses manifestiert sich in einem Zufriedenheitsurteil, das zukünftiges Verhalten beeinflusst.

Bhattacharjee [3] erweitert dieses Modell für den IT-Kontext und zeigt, dass Zufriedenheit die Fortsetzungsabsicht (Continuance Intention) maßgeblich beeinflusst – ein Aspekt, der für Online-Shops mit ihrer hohen Wechselbarriere besonders relevant ist.

b) E-Customer Satisfaction:

E-Customer Satisfaction unterscheidet sich in mehreren Aspekten von traditioneller Kundenzufriedenheit [1]:

Mehrdimensionalität: Online-Zufriedenheit umfasst nicht nur das Produkt selbst, sondern auch Website-Usability, Checkout-Prozess, Lieferung, Verpackung und After-Sales-Service.

Fehlende haptische Erfahrung: Kunden können Produkte vor dem Kauf nicht physisch prüfen, was die Rolle von Produktbeschreibungen und Bewertungen anderer Kunden verstärkt.

Zeitliche Entkopplung: Zwischen Bestellung und Erhalt vergehen Tage, wodurch mehrere Touchpoints entstehen, die jeweils zur Gesamtzufriedenheit beitragen.

Soziale Komponente: Online-Bewertungen und Ratings werden zu einem integralen Bestandteil des Kaufprozesses und beeinflussen sowohl die Erwartungsbildung als auch die Wahrnehmung anderer Kunden.

McKinney et al. [6] betonen, dass Web-basierte Kundenzufriedenheit stark von der Qualität der Information und des Interaktionsdesigns abhängt. Ihre Forschung zeigt, dass technische Faktoren (Ladezeiten, Navigation, Fehlerfreiheit) einen direkten Einfluss auf die Gesamtzufriedenheit haben.

c) Messung der E-Service-Qualität:

Parasuraman et al. [8] entwickelten das E-S-QUAL Framework, ein speziell für elektronische Services konzipiertes Messinstrument. Es umfasst vier Dimensionen:

Efficiency: Die Leichtigkeit und Geschwindigkeit, mit der Kunden auf die Website zugreifen und diese nutzen können.

Fulfillment: Das Ausmaß, in dem Versprechen bezüglich Lieferung und Verfügbarkeit eingehalten werden.

System Availability: Die korrekte technische Funktionalität der Website.

Privacy: Der Schutz von Kundendaten und die Sicherheit der Transaktionen.

Wolfenbarger und Gilly [9] ergänzen mit eTailQ ein weiteres Framework, das besonders die Dimensionen Website Design, Customer Service, Privacy/Security und Fulfillment/Reliability betont.

d) Net Promoter Score (NPS):

Der Net Promoter Score [10] hat sich als praktische, einfach zu implementierende Metrik etabliert. Kunden werden gefragt: "Wie wahrscheinlich ist es, dass Sie unser Unternehmen/Produkt einem Freund oder Kollegen weiterempfehlen?" (Skala 0-10).

Basierend auf der Antwort werden Kunden kategorisiert:

- Promotoren (9-10): Begeisterte Kunden, die aktiv werben
- Passive (7-8): Zufriedene, aber nicht begeisterte Kunden
- Detraktoren (0-6): Unzufriedene Kunden, die negativ kommunizieren

Der NPS wird berechnet als: $NPS = \% \text{ Promotoren} - \% \text{ Detraktoren}$

Trotz methodischer Kritik hat sich der NPS aufgrund seiner Einfachheit und Korrelation mit Geschäftserfolg durchgesetzt und ist eine der am häufigsten verwendeten Metriken im E-Commerce.

C. Technische Grundlagen

a) Service-orientierte Architektur (SOA):

Service-orientierte Architekturen strukturieren Anwendungen als Sammlung lose gekoppelter Services, die über definierte Schnittstellen kommunizieren. Die Kernprinzipien von SOA sind für CuSaSy relevant:

Loose Coupling: Services sind unabhängig voneinander und können separat entwickelt, deployt und skaliert werden.

Service Contract: Jeder Service definiert einen klaren Vertrag (API-Spezifikation), der seine Funktionalität beschreibt.

Autonomy: Services haben Kontrolle über ihre eigene Logik und Daten.

Reusability: Services sind so gestaltet, dass sie von verschiedenen Konsumenten genutzt werden können.

Statelessness: Services minimieren die Speicherung von Zustandsinformationen, um Skalierbarkeit zu ermöglichen.

Im Kontext von CuSaSy bedeutet dies, dass das Zufriedenheitstracking als eigenständiger Service konzipiert wird, der von verschiedenen E-Commerce-Plattformen genutzt werden kann, ohne tiefe Integration in deren Systeme zu erfordern.

b) RESTful API Design:

REST (Representational State Transfer) ist ein Architekturstil für verteilte Systeme, der besonders für Web-APIs verbreitet ist. REST basiert auf folgenden Prinzipien:

Resource-Orientierung: Alles wird als Ressource modelliert (z.B. Bewertungen, Produkte, Kunden)

HTTP-Methoden: Standard-HTTP-Verben definieren Operationen (GET zum Lesen, POST zum Erstellen, PUT/PATCH zum Aktualisieren, DELETE zum Löschen)

Stateless Communication: Jede Anfrage enthält alle notwendigen Informationen, der Server hält keinen Client-State

Uniform Interface: Konsistente, vorhersehbare API-Struktur

RESTful APIs eignen sich besonders für die Integration von CuSaSy in bestehende E-Commerce-Systeme, da sie sprachunabhängig und einfach zu konsumieren sind.

c) Zugriffskontrolle und Sicherheit:

Role-Based Access Control (RBAC):

RBAC ist ein bewährtes Modell zur Zugriffskontrolle, bei dem Berechtigungen nicht direkt Benutzern, sondern Rollen zugewiesen werden. Benutzer erhalten dann eine oder mehrere Rollen.

Die Grundkonzepte sind:

Users (Benutzer): Individuelle Personen oder Systeme, die auf das System zugreifen

Roles (Rollen): Gruppierungen von Berechtigungen, die typischen Tätigkeitsprofilen entsprechen (z.B. "Administrator", "Analyst", "Viewer")

Permissions (Berechtigungen): Spezifische Operationen auf Ressourcen (z.B. "Bewertungen lesen", "Bewertungen löschen", "Statistiken exportieren")

Sessions (Sitzungen): Temporäre Zuordnungen von Benutzern zu aktivierten Rollen

Für CuSaSy sind typische Rollen:

- **Administrator:** Volle Kontrolle über System-Konfiguration und Benutzerverwaltung
- **Manager:** Zugriff auf alle Daten des eigenen Mandanten, kann Berichte erstellen und exportieren
- **Analyst:** Lesezugriff auf Daten und Statistiken, keine Schreibrechte
- **API User:** Technischer Benutzer für System-zu-System-Integration

Attribute-Based Access Control (ARBAC):

ARBAC erweitert RBAC um attributbasierte Zugriffskontrolle und bietet feinere Granularität. Anstatt nur auf Rollen zu basieren, werden Zugriffsentscheidungen basierend auf Attributen getroffen:

Benutzerattribute: Eigenschaften des Benutzers (Abteilung, Standort, Sicherheitsfreigabe)

Ressourcenattribute: Eigenschaften der zu schützenden Daten (Vertraulichkeitsstufe, Produktkategorie, Zeitraum)

Umgebungsattribute: Kontextuelle Faktoren (Tageszeit, Netzwerkstandort, verwendetes Gerät)

Policies: Regeln, die definieren, unter welchen Bedingungen Zugriff gewährt wird

Ein ARBAC-System für CuSaSy könnte beispielsweise folgende Policy implementieren:

ERLAUBEN, wenn:

- Benutzerrolle = "Manager" UND
- Benutzer.Mandant = Bewertung.Mandant UND
- (Aktuelle_Zeit zwischen 08:00 und 18:00 ODER Benutzer.VPN_Zugriff = true)

ARBAC bietet mehrere Vorteile für Multi-Tenant-Systeme wie CuSaSy:

Mandantenisolation: Automatische Sicherstellung, dass Benutzer nur auf Daten ihres eigenen Mandanten zugreifen können

Dynamische Anpassung: Zugriffsentscheidungen können sich zur Laufzeit ändern, ohne Rollendefinitionen zu modifizieren

Compliance: Einfachere Umsetzung regulatorischer Anforderungen (z.B. DSGVO-konforme Zugriffsprotokolle)

Feingranulare Kontrolle: Zugriff kann auf spezifische Datensätze oder Zeiträume beschränkt werden

Die Implementierung von ARBAC erfordert jedoch zusätzliche Infrastruktur:

- 1) **Policy Decision Point (PDP):** Komponente, die Zugriffsentscheidungen trifft
- 2) **Policy Enforcement Point (PEP):** Komponente, die Entscheidungen durchsetzt
- 3) **Policy Information Point (PIP):** Komponente, die Attribute bereitstellt
- 4) **Policy Administration Point (PAP):** Verwaltungsinterface für Policies

Für CuSaSy wird ein hybrides Modell vorgeschlagen: RBAC für grundlegende Berechtigungen kombiniert mit attributbasierten Regeln für Mandantenisolation und zeitliche Zugriffseinschränkungen. Dies bietet ein ausgewogenes Verhältnis zwischen Sicherheit, Flexibilität und Implementierungsaufwand.

d) *Authentication und JSON Web Tokens (JWT):*

In Microservice-Umgebungen ist die Authentifizierung besonders herausfordernd, da mehrere Services validieren müssen, ob eine Anfrage autorisiert ist, ohne einen zentralen Session-Store zu benötigen.

JSON Web Tokens (JWT) lösen dieses Problem durch selbstbeschreibende, signierte Tokens. Ein JWT besteht aus drei Teilen:

Header: Definiert den Token-Typ und Signaturalgorithmus
Payload: Enthält Claims (Aussagen über den Benutzer), z.B. Benutzer-ID, Rollen, Ablaufzeit
Signature: Kryptographische Signatur zur Verifikation der Integrität

Vorteile von JWT für CuSaSy:

- **Stateless:** Keine serverseitige Session-Speicherung notwendig
- **Skalierbar:** Services können Tokens unabhängig validieren
- **Standardisiert:** Breite Tool- und Library-Unterstützung
- **Flexible Claims:** Können Mandanten-ID, Rollen und weitere Attribute enthalten

Sicherheitsaspekte:

- Tokens sollten über HTTPS übertragen werden
- Kurze Ablaufzeiten (z.B. 15 Minuten) mit Refresh-Token-Mechanismus
- Sensitive Daten sollten nicht im Payload gespeichert werden (da Base64-kodiert, aber nicht verschlüsselt)
- Secret Keys müssen sicher verwaltet werden

e) *Multi-Tenancy Patterns:*

Multi-Tenancy bezeichnet die Fähigkeit eines Systems, mehrere Mandanten (Tenants) auf derselben Infrastruktur zu bedienen, während deren Daten strikt isoliert bleiben. Es gibt verschiedene Isolation-Strategien:

Separate Datenbanken: Jeder Mandant erhält eine eigene Datenbank. Höchste Isolation, aber höhere Infrastrukturkosten.

Shared Database, Separate Schemas: Gemeinsame Datenbank, aber separate Schemas pro Mandant. Guter Kompromiss zwischen Isolation und Effizienz.

Shared Schema mit Tenant-ID: Alle Mandanten teilen dasselbe Datenbankschema, Zeilen werden durch Tenant-ID

unterschieden. Höchste Effizienz, erfordert aber sorgfältige Implementierung zur Vermeidung von Datenlecks.

Für CuSaSy wird das Shared Schema Pattern mit strikter Row-Level-Security empfohlen, um Skalierbarkeit zu maximieren bei gleichzeitiger Sicherstellung der Datenisolation.

D. Zusammenfassung

Dieses Kapitel hat die theoretischen und technischen Grundlagen für CuSaSy etabliert. Die Erkenntnisse aus E-Commerce-Forschung, insbesondere die Bedeutung der Kundenzufriedenheit und deren Messung, motivieren die Entwicklung eines spezialisierten Systems. Die vorgestellten technischen Konzepte – SOA, REST, RBAC/ARBAC, JWT und Multi-Tenancy – bilden das Fundament für die Architekturentscheidungen, die in Kapitel IV detailliert werden. Die Kombination aus theoretisch fundierten Zufriedenheitsmodellen und modernen Architekturpatterns ermöglicht ein System, das sowohl wissenschaftlich fundiert als auch praktisch einsetzbar ist.

III. SYSTEMDESIGN UND TECHNISCHE DOKUMENTATION

Dieses Kapitel beschreibt die Architektur und Funktionsweise des Customer Satisfaction Systems aus einer anwendungsorientierten Perspektive. Es wird dargelegt, wie das System aufgebaut ist, welche Funktionalitäten es bietet und wie Benutzer mit dem System interagieren. Die Dokumentation richtet sich primär an Administratoren, Analysten und Stakeholder, die das System verstehen, nutzen oder in ihre bestehende Infrastruktur integrieren möchten.

A. Architekturüberblick

CuSaSy folgt einer modernen drei Schichten-Architektur, die Frontend, Backend und Datenspeicherung klar voneinander trennt. Diese Trennung ermöglicht es, dass jede Komponente unabhängig entwickelt, gewartet und skaliert werden kann. Die Architektur ist bewusst einfach gehalten, um Wartbarkeit und Verständlichkeit zu maximieren, ohne dabei Flexibilität und Erweiterbarkeit zu opfern.

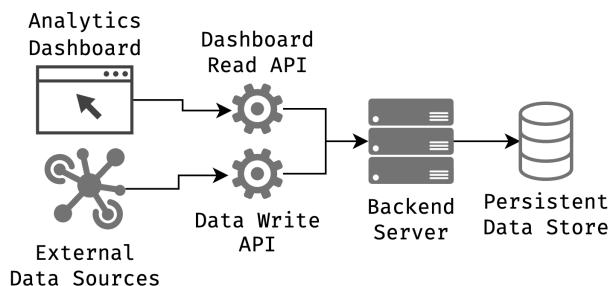


Fig. 1. Systemarchitektur von CuSaSy mit Frontend, Backend-APIs und Datenspeicherung

Das Frontend besteht aus einer webbasierten Analytics-Dashboard-Anwendung, die Benutzern ermöglicht, Kundenzufriedenheitsdaten zu visualisieren und zu analysieren. Benutzer greifen über einen Webbrowser auf das Dashboard zu und können interaktiv Zeiträume auswählen, Daten filtern und detaillierte Berichte einsehen. Das Dashboard

ist responsiv gestaltet und passt sich automatisch an verschiedene Bildschirmgrößen an, von Desktop-Monitoren bis zu Tablet-Geräten.

Das Backend fungiert als zentrale Verarbeitungseinheit und stellt zwei spezialisierte API-Schnittstellen bereit. Die Dashboard Read API ist optimiert für den Abruf von Daten durch das Frontend. Sie liefert aggregierte Statistiken, Zeitreihendaten für Charts und detaillierte Tabellendaten. Diese API ist so konzipiert, dass häufig abgefragte Daten effizient bereitgestellt werden können. Die Data Write API hingegen dient zur Erfassung neuer Bewertungen von externen Systemen. E-Commerce-Plattformen können nach Abschluss einer Bestellung oder nach Auslieferung eines Produkts automatisch eine Bewertungsanfrage über diese API senden.

Die Persistent Data Store-Komponente speichert alle Bewertungen, Kundendaten und zugehörige Metadaten dauerhaft. Aktuell nutzt das System eine JSON-basierte Dateiablage für Entwicklungszwecke, ist aber so konzipiert, dass eine Migration zu einer relationalen Datenbank wie PostgreSQL mit minimalem Aufwand möglich ist. Die Datenbank ist das einzige zustandsbehaftete Element im System; Frontend und Backend sind stateless konzipiert.

External Data Sources repräsentieren die E-Commerce-Systeme und anderen Quellsysteme, die Kundenfeedback an CuSaSy übermitteln. Diese Systeme können über die Write API Bewertungen einreichen, ohne dass eine tiefe Integration erforderlich ist. Die Kommunikation erfolgt über standardisierte HTTP-Requests mit JSON-Payloads.

B. Das Analytics Dashboard

Das Dashboard ist die primäre Benutzeroberfläche von CuSaSy und richtet sich an Analysten, Manager und andere Stakeholder, die Einblicke in die Kundenzufriedenheit gewinnen möchten. Es ist in zwei Hauptbereiche unterteilt: die Overview-Ansicht für aggregierte Zeitreihendaten und die Details-Ansicht für granulare Datenanalyse.

a) Overview-Bereich:

Die Overview-Seite bietet einen umfassenden Überblick über die wichtigsten Key Performance Indicators der Kundenzufriedenheit. Beim Öffnen des Dashboards werden automatisch Daten für die letzten 30 Tage geladen und visuell aufbereitet. Benutzer sehen sofort die wesentlichen Trends und können auf einen Blick erkennen, ob die Kundenzufriedenheit steigt, stagniert oder sinkt.

Der obere Bereich der Overview-Seite präsentiert drei KPI-Cards, die aktuelle Nutzungsstatistiken zusammenfassen. Die erste Card zeigt Nutzungsmetriken wie die Anzahl eingereicherter Bewertungen, genehmigte Reviews und durchschnittliche Ratings. Für jede Metrik wird der aktuelle Wert angezeigt sowie die prozentuale Veränderung gegenüber dem Vergleichszeitraum. Ein farbcodiertes Badge signalisiert auf einen Blick, ob die Entwicklung positiv oder negativ ist: Grüne Badges zeigen Verbesserungen, rote Verschlechterungen an.

Die zweite Card fokussiert auf Workspace-Metriken und zeigt Informationen wie die Anzahl aktiver Kunden, wöchentlich aktive Nutzer und System-Uptime. Diese

Metriken geben Administratoren Einblick in die operative Gesundheit des Systems. Die dritte Card visualisiert Kostenverteilungen mittels eines horizontalen Balkendiagramms. Verschiedene Kostenkategorien werden proportional dargestellt, sodass Benutzer sofort erkennen, welche Bereiche die höchsten Aufwände verursachen.

Unterhalb der KPI-Cards befindet sich die Filterbar, ein zentrales Steuerungselement des Dashboards. Über einen Date Picker können Benutzer präzise Zeiträume auswählen. Das System unterstützt sowohl vordefinierte Bereiche wie "Letzte 7 Tage" oder "Letzter Monat" als auch vollständig benutzerdefinierte Datumsbereiche. Neben der Zeitraumauswahl können Benutzer einen Vergleichsmodus wählen. Der Modus "Previous Period" vergleicht den gewählten Zeitraum mit der unmittelbar vorhergehenden Periode gleicher Länge. "Last Year" vergleicht mit dem gleichen Zeitraum im Vorjahr. "No Comparison" deaktiviert den Vergleich und zeigt nur absolute Werte.

Die Kategorie-Auswahl ermöglicht es, spezifische Metriken ein- oder auszublenden. Standardmäßig sind alle verfügbaren Metriken aktiviert, aber Benutzer können durch Anklicken von Checkboxes irrelevante Metriken ausblenden. Dies ist besonders nützlich bei Präsentationen oder wenn man sich auf spezifische Aspekte konzentrieren möchte.

Der Hauptbereich der Overview-Seite zeigt für jede ausgewählte Metrik eine interaktive Chart-Card. Jede Card präsentiert ein Liniendiagramm, das die zeitliche Entwicklung der Metrik visualisiert. Die X-Achse repräsentiert die Zeit in täglichen Intervallen, die Y-Achse den Wert der Metrik. Wenn ein Vergleichsmodus aktiviert ist, zeigt das Diagramm zwei Linien: eine in Indigo für den aktuellen Zeitraum, eine in Grau für den Vergleichszeitraum. Dies ermöglicht intuitive visuelle Vergleiche.

Oberhalb jedes Charts werden der aggregierte Gesamtwert für den Zeitraum und die prozentuale Veränderung angezeigt. Beim Hovern über das Diagramm erscheint ein Tooltip, der die exakten Werte für den jeweiligen Tag zeigt. Dies ermöglicht Benutzern, Anomalien oder besondere Ereignisse an spezifischen Tagen zu identifizieren. Die Charts sind so optimiert, dass auch bei Zeiträumen von mehreren Monaten die Performance flüssig bleibt.

Das System erfasst und visualisiert verschiedene Metriken, die aus den Bewertungsdaten abgeleitet werden. "Reviews submitted" zählt die Anzahl neu eingegangener Bewertungen pro Tag. "Reviews approved" zeigt, wie viele Bewertungen nach Prüfung freigeschaltet wurden. "Average rating" berechnet den Durchschnitt aller Bewertungen eines Tages. "Customer interactions" summiert alle Kundenkontakte, während "Support escalations" die Anzahl problematischer Bewertungen mit niedrigen Ratings erfasst, die möglicherweise Support-Intervention erfordern.

b) Details-Bereich:

Der Details-Bereich bietet eine tabellarische Ansicht aller Bewertungen mit umfangreichen Filter-, Such- und Sortierfunktionen. Während die Overview für strategische Analyse konzipiert ist, erlaubt die Details-Ansicht operative Arbeit mit individuellen Datensätzen. Administratoren und Support-Mitarbeiter nutzen diese Ansicht, um spezifische

Bewertungen zu finden, zu prüfen und gegebenenfalls Maßnahmen einzuleiten.

Die Tabelle zeigt standardmäßig zwanzig Bewertungen pro Seite und bietet Pagination-Kontrollen zur Navigation durch größere Datenmengen. Jede Zeile repräsentiert eine einzelne Bewertung mit folgenden Spalten: Owner (Kundenname), Status (pending, approved, rejected), Rating (1-5 Sterne), Region (geografischer Standort), Costs (geschätzte Kosten), Stability (abgeleitet vom Rating: Stable, Warning, Critical) und Last Edited (letzter Änderungszeitpunkt).

Die Filterbar oberhalb der Tabelle ermöglicht Mehrfach-Filterung nach verschiedenen Kriterien. Benutzer können nach Status filtern, um beispielsweise nur ausstehende Bewertungen anzuzeigen. Ein Rating-Filter erlaubt Fokussierung auf spezifische Bewertungsstufen. Regional-Filter helfen bei der Analyse geografischer Unterschiede in der Kundenzufriedenheit. Eine Freitextsuche durchsucht Kundennamen, Bewertungstitel und Kommentare simultan.

Alle Spaltenüberschriften sind klickbar und ermöglichen Sortierung in aufsteigender oder absteigender Reihenfolge. Ein kleines Pfeilsymbol in der aktiven Spalte zeigt die aktuelle Sortierrichtung. Benutzer können beispielsweise nach Rating sortieren, um die schlechtesten Bewertungen zuerst zu sehen und prioritär zu bearbeiten. Sortierung nach Datum hilft bei chronologischer Aufarbeitung.

Die Tabelle unterstützt Zeilenauswahl durch Anklicken. Ausgewählte Zeilen werden visuell hervorgehoben durch einen farbigen linken Rand und einen leicht geänderten Hintergrund. Dies ermöglicht Bulk-Operationen auf mehreren Bewertungen gleichzeitig. Ein Bulk-Editor erscheint am unteren Bildschirmrand, sobald mindestens eine Zeile ausgewählt ist. Über diesen Editor können Benutzer den Status mehrerer Bewertungen gleichzeitig ändern, was besonders bei der Freischaltung großer Mengen hilft.

Die Tabelle ist vollständig responsiv und passt sich an kleinere Bildschirme an, indem weniger wichtige Spalten ausgeblendet oder in ein Drawer-Menü verschoben werden. Auf Tablets im Hochformat beispielsweise werden nur Owner, Rating und Status direkt angezeigt, während weitere Details durch Antippen einer Zeile expandiert werden.

C. Backend-API-Schnittstellen

Das Backend stellt mehrere REST-API-Endpunkte bereit, die sowohl vom Frontend als auch von externen Systemen genutzt werden. Alle API-Endpunkte folgen konsistenten Konventionen bezüglich Request-Format, Response-Struktur und Fehlerbehandlung. Das zugrundeliegende Datenmodell, welches die Beziehungen zwischen Marktplätzen, Produkten und Bewertungen definiert, ist in Abbildung 3 dargestellt.

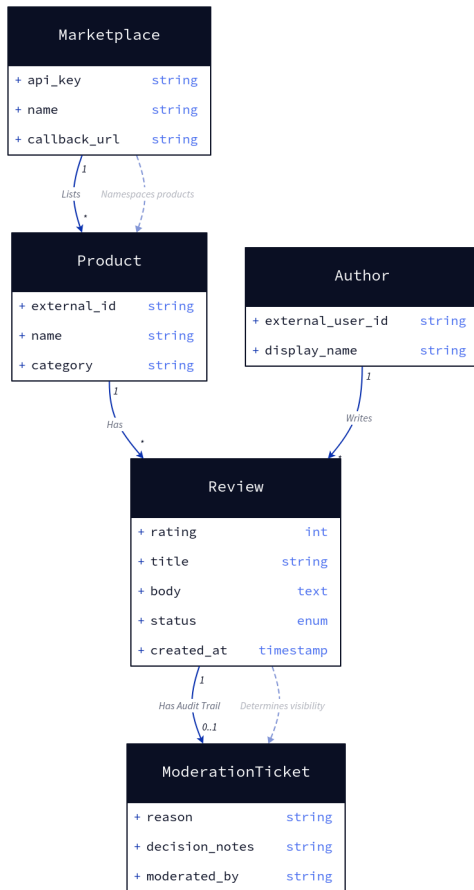


Fig. 2. Datenmodell und Entitäten-Beziehungen

a) Reviews-API für Bewertungsmanagement:

Die Reviews-API ermöglicht vollständiges CRUD-Management von Kundenbewertungen. Der GET-Endpunkt auf `/api/reviews` liefert eine Liste aller Bewertungen mit optionaler Filterung und Pagination. Benutzer können nach Status filtern, um nur genehmigte, ausstehende oder abgelehnte Bewertungen zu erhalten. Ein Rating-Filter erlaubt Eingrenzung auf spezifische Bewertungsstufen. Customer-ID-Filter ermöglichen das Auffinden aller Bewertungen eines bestimmten Kunden. Limit und Offset-Parameter unterstützen Pagination für große Datenmengen.

Zum Abrufen einer einzelnen Bewertung wird GET `/api/reviews/:id` verwendet, wobei `:id` die eindeutige Identifikation der Bewertung ist. Dies liefert alle Details inklusive Kundennamen, Rating, Titel, Kommentar, Zeitstempel und Status. Falls die ID nicht existiert, wird ein 404-Fehler mit aussagekräftiger Fehlermeldung zurückgegeben.

Das Erstellen neuer Bewertungen erfolgt über POST `/api/reviews` mit einem JSON-Body, der alle erforderlichen Felder enthält. Pflichtfelder sind `customerId` zur Identifikation des bewertenden Kunden, `customerName` für die Anzeige, `rating` als numerischer Wert zwischen eins und fünf, `title` als Überschrift der Bewertung und `comment` als Freitext-Bewertung.

Optional kann ein Status übergeben werden; fehlt dieser, wird automatisch "pending" gesetzt. Das System validiert alle Eingaben und gibt bei fehlerhaften Daten einen 400-Fehler mit detaillierten Validierungsfehlern zurück.

Aktualisierungen existierender Bewertungen nutzen PUT `/api/reviews/:id`. Es können beliebige Teilmengen der Felder übermittelt werden; nur die übergebenen Felder werden geändert. Dies erlaubt beispielsweise reine Status-Updates ohne Änderung der eigentlichen Bewertung. Das System merged die neuen Werte mit den bestehenden und aktualisiert den `updatedAt-Timestamp` automatisch.

Das Löschen von Bewertungen erfolgt über DELETE `/api/reviews/:id`. Nach erfolgreicher Löschung wird ein Bestätigungsstatus zurückgegeben. Gelöschte Bewertungen sind permanent entfernt und können nicht wiederhergestellt werden, weshalb diese Operation typischerweise nur Administratoren vorbehalten ist.

Alle Responses folgen einem einheitlichen Format mit einem `success-Boolean-Feld`, das anzeigt, ob die Operation erfolgreich war. Bei Erfolg enthält das `data-Feld` die angefragten Daten oder das Ergebnis der Operation. Bei Fehlern enthält das `error-Feld` eine nutzerfreundliche Fehlerbeschreibung, während ein optionales `details-Feld` technische Details oder Validierungsfehler auflistet.

b) Metrics-API für Dashboard-Daten:

Die Metrics-API ist speziell für die Befüllung des Dashboards mit aggregierten und zeitbasierten Daten konzipiert. Der Hauptendpunkt GET `/api/metrics/timeseries` liefert Zeitreihendaten für Chart-Darstellungen. Benutzer spezifizieren einen Zeitraum über `startDate` und `endDate` Parameter im YYYY-MM-DD Format. Das System generiert für jeden Tag im Zeitraum einen Datenpunkt mit allen relevanten Metriken.

Die zurückgegebenen Daten enthalten Arrays von Objekten, wobei jedes Objekt ein Datum und die zugehörigen Metrikwerte repräsentiert. Beispielsweise "Reviews submitted" zeigt die Anzahl neu eingegangener Bewertungen, "Average rating" den Tagesdurchschnitt aller Ratings, "Support escalations" die Anzahl problematischer Bewertungen mit Rating kleiner oder gleich zwei. Diese Struktur ist direkt kompatibel mit den Chart-Komponenten des Frontends.

Der metrics-Parameter ermöglicht selektives Abrufen nur bestimmter Metriken. Dies reduziert Datenvolumen und Verarbeitungszeit, wenn nur spezifische Charts angezeigt werden sollen. Der Request `/api/metrics/timeseries?metrics=Reviews submitted,Average rating` liefert beispielsweise nur diese zwei Metriken für jeden Tag.

Der Summary-Endpunkt GET `/api/metrics/summary` aggregiert Daten über einen Zeitraum und berechnet KPIs sowie Vergleiche. Er liefert `totalReviews` als Gesamtanzahl, `averageRating` als Durchschnittswert, `satisfactionScore` als Prozentsatz der vier- und fünf-Sterne-Bewertungen. `statusBreakdown` zählt Bewertungen nach Status, `ratingBreakdown` nach Sternbewertung. `customerSentiment` kategorisiert Bewertungen in positive, neutrale und negative basierend auf Rating-Schwellwerten.

Bei aktiviertem Vergleichsmodus berechnet die API automatisch einen Vergleichszeitraum. Im "previous_period"-

Modus wird ein Zeitraum gleicher Länge unmittelbar vor dem angefragten Zeitraum gewählt. Im “last_year”-Modus wird der exakt gleiche Kalenderzeitraum ein Jahr zuvor verwendet. Für beide Zeiträume werden identische Metriken berechnet und prozentuale Veränderungen ermittelt. Diese Informationen befüllen die KPI-Cards im Dashboard mit Trend-Badges.

c) Usage-API für detaillierte Daten:

Die Usage-API unter /api/usage/details ist für die Befüllung der Details-Tabelle optimiert. Sie unterstützt umfangreiche Filterung, Sortierung und Pagination. Filter können kombiniert werden: status, rating, customerId, owner (Kundenname), region, stability und sentiment können gleichzeitig angewendet werden. Ein search-Parameter ermöglicht Volltextsuche über Titel, Kommentare und Kundennamen.

Die Sortierung erfolgt über sortBy und sortOrder Parameter. sortBy akzeptiert jeden Spaltennamen wie “rating”, “createdAt” oder “owner”. sortOrder kann “asc” für aufsteigend oder “desc” für absteigend sein. Standardmäßig wird nach createdAt absteigend sortiert, sodass neueste Bewertungen zuerst erscheinen.

Pagination wird über limit und offset realisiert. limit bestimmt die Anzahl Datensätze pro Seite, typischerweise zwanzig oder fünfzig. offset gibt an, wie viele Datensätze übersprungen werden sollen. Für Seite zwei mit limit=20 wäre offset=20. Die Response enthält ein pagination-Objekt mit total (Gesamtanzahl passender Datensätze), limit, offset und hasMore (Boolean, ob weitere Seiten existieren).

Zusätzlich zu den eigentlichen Datensätzen liefert die API ein statistics-Objekt mit aggregierten Informationen über das gefilterte Ergebnis. Dies beinhaltet statusBreakdown, sentimentBreakdown, stabilityBreakdown, regionBreakdown und averageRating. Diese Statistiken ermöglichen es dem Frontend, Zusammenfassungen anzuzeigen wie “Zeige 15 von 200 Bewertungen (Durchschnitt: 4.2 Sterne)”.

Die API mappt interne Review-Daten auf Dashboard-freundliche Formate. Beispielsweise wird aus dem numerischen Rating ein Sentiment-Label abgeleitet: Ratings vier und fünf werden zu “Positive”, drei zu “Neutral”, eins und zwei zu “Negative”. Ähnlich wird Stability berechnet: Hohe Ratings werden “Stable”, mittlere “Warning”, niedrige “Critical”. Dies vereinfacht Filterung und visuelle Darstellung im Dashboard.

d) Export-API für Datenintegration:

Die Export-API ermöglicht das Exportieren von Bewertungsdaten in verschiedenen Formaten für Integration mit externen Systemen wie Data Warehouses oder Business Intelligence Tools. Der Endpunkt /api/export/csv generiert eine CSV-Datei mit allen oder gefilterten Bewertungen. Die CSV enthält Spaltenüberschriften und kann direkt in Excel, Google Sheets oder Datenbankimport-Tools geladen werden. Alle Filter-Parameter der Reviews-API werden unterstützt, sodass nur relevante Daten exportiert werden.

Der JSON-Export unter /api/export/json liefert Bewertungsdaten im JSON-Format, ideal für programmatische Weiterverarbeitung oder API-zu-API-Integration. Die Struktur ist identisch zur Reviews-API, aber als downloadbare Datei verpackt mit korrektem Content-Disposition Header.

Ein besonderer Endpunkt /api/export/summary liefert eine kompakte Zusammenfassung aller Statistiken ohne einzelne Bewertungen. Dies ist nützlich für regelmäßige Reports oder Dashboard-Integration in anderen Systemen. Die Summary enthält totalReviews, averageRating, statusBreakdown und ratingBreakdown sowie einen generatedAt-Timestamp für Versionierung.

D. Datenfluss und Interaktion

Das Zusammenspiel der Komponenten lässt sich anhand typischer Nutzungsszenarien verdeutlichen. Abbildung 2 zeigt die zentralen Akteure und ihre Interaktionen mit dem System.

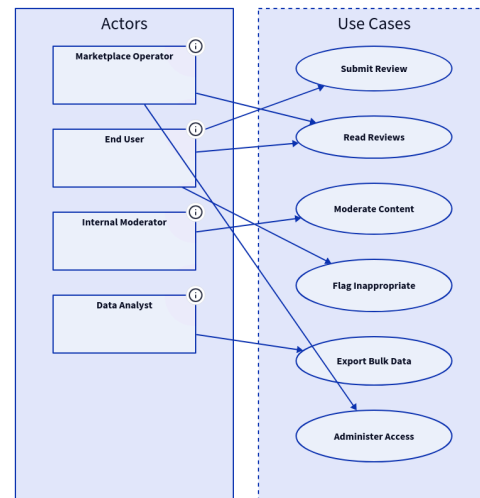


Fig. 3. Use-Case-Diagramm der zentralen Systeminteraktionen

Wenn ein Benutzer das Dashboard öffnet, sendet das Frontend automatisch mehrere API-Requests. Zunächst wird /api/metrics/timeseries für die letzten dreißig Tage abgerufen, um die Overview-Charts zu befüllen. Parallel dazu wird /api/metrics/summary für KPI-Cards angefragt. Diese initialen Requests erfolgen gleichzeitig, um Ladezeit zu minimieren.

Wählt der Benutzer einen anderen Zeitraum über den Date Picker, werden die API-Calls mit den neuen Datumsparemtern wiederholt. Das Frontend zeigt während des Ladevorgangs einen subtilen Ladeindikator. Sobald die Daten eintreffen, werden die Charts flüssig neu gerendert mit Animationen, die Änderungen visuell hervorheben. Bei aktiviertem Vergleichsmodus fordert das Frontend automatisch Daten für beide Zeiträume an und visualisiert sie parallel.

Beim Wechsel zur Details-Ansicht wird /api/usage/details mit Standard-Pagination aufgerufen. Ändert der Benutzer Filter oder Sortierung, wird ein neuer Request mit aktualisierten Parametern gesendet. Das System implementiert Debouncing bei der Textsuche, sodass nicht bei jedem Tastendruck ein Request erfolgt, sondern erst nach einer kurzen Pause. Dies reduziert Server-Last und verbessert Responsiveness.

Für externe Systeme, die Bewertungen einreichen möchten, ist der Ablauf wie folgt: Nach Abschluss einer Transaktion im E-Commerce-System sendet dieses einen POST-Request an `/api/reviews` mit Kunden- und Bestelldaten. Das Backend validiert die Daten, erstellt eine neue Bewertung mit Status "pending" und speichert sie persistent. Eine optionale Bestätigungs-E-Mail könnte an den Kunden gesendet werden mit Link zur Bewertungsabgabe. Sobald der Kunde die Bewertung verfasst, wird sie via PUT aktualisiert.

Administratoren können über die Details-Tabelle Bewertungen sichten. Sie wählen mehrere pending-Bewertungen aus und nutzen den Bulk-Editor, um sie auf "approved" zu setzen. Das Frontend sendet für jede Bewertung einen PUT-Request. Nach erfolgreicher Aktualisierung werden die Zeilen in der Tabelle automatisch aktualisiert, um den neuen Status zu reflektieren. Die Overview-Metriken werden bei nächster Aktualisierung die genehmigten Bewertungen berücksichtigen.

E. Sicherheit und Zugriffskontrolle

Obwohl das aktuelle System noch keine vollständige Authentifizierung implementiert, ist die Architektur darauf vorbereitet. Das in Kapitel II beschriebene JWT-basierte Authentifizierungskonzept kann nahtlos integriert werden. In einer produktiven Umgebung würde jeder API-Request einen Authorization-Header mit einem gültigen JWT enthalten. Das Backend würde diesen Token validieren und die enthaltenen Claims nutzen, um rollenbasierte Zugriffskontrolle durchzusetzen.

Verschiedene Benutzerrollen hätten unterschiedliche Berechtigungen. Viewer könnten nur das Dashboard sehen und Daten abrufen, aber keine Änderungen vornehmen. Analysts hätten zusätzlich Export-Rechte. Managers könnten Bewertungen genehmigen oder ablehnen. Administratoren hätten volle Kontrolle inklusive Löschrechte und Benutzerverwaltung. Diese Rollen würden im JWT als Claim kodiert und bei jedem Request überprüft.

Für Multi-Tenancy-Szenarien, wo mehrere Marktplätze das gleiche CuSaSy-System nutzen, würde der JWT eine tenant-ID enthalten. Alle Datenbank-Queries würden automatisch nach dieser ID filtern, sodass Benutzer nur Daten ihres eigenen Mandanten sehen. Dies gewährleistet strikte Datenisolation ohne komplexe Logik in der Anwendungsschicht.

Die CORS-Konfiguration des Backends würde in Produktion restriktiv gesetzt, um nur Requests von autorisierten Origins zu akzeptieren. Rate Limiting würde vor Missbrauch schützen, indem die Anzahl Requests pro IP-Adresse oder Token begrenzt wird. Sensitive Operationen wie Löschungen würden zusätzliche Bestätigung erfordern.

F. Erweiterbarkeit und Integration

Die modulare Architektur von CuSaSy erleichtert Erweiterungen. Neue Metriken können hinzugefügt werden, indem die Berechnungslogik im Backend erweitert und entsprechende Chart-Cards im Frontend erstellt werden. Das System ist so konzipiert, dass solche Änderungen keine Modifikation der Kernarchitektur erfordern.

Integration mit externen Systemen erfolgt primär über die REST-APIs. E-Commerce-Plattformen können Webhooks konfigurieren, die bei bestimmten Events die Write-API aufrufen. CRM-Systeme können regelmäßig die Export-API nutzen, um Bewertungsdaten zu synchronisieren. Business Intelligence Tools können die Metrics-API anzapfen, um Kundenzufriedenheit in umfassendere Dashboards einzubinden.

Für Echtzeit-Updates könnte das System um WebSocket-Support erweitert werden. Das Dashboard würde eine WebSocket-Verbindung öffnen und bei jeder neuen Bewertung eine Push-Benachrichtigung erhalten. Dies würde Live-Updates ohne manuellen Reload ermöglichen. Die Grundarchitektur unterstützt solche Erweiterungen ohne fundamentale Änderungen.

Das Frontend ist komponentenbasiert aufgebaut, was Anpassungen erleichtert. Unternehmen können das Farbschema ändern, ihr Logo integrieren oder zusätzliche Visualisierungen hinzufügen, ohne das gesamte System neu schreiben zu müssen. Die Verwendung von TypeScript gewährleistet, dass solche Änderungen typischer bleiben und Fehler frühzeitig erkannt werden.

G. Deployment und Betrieb

CuSaSy kann in verschiedenen Umgebungen deployt werden. Für Entwicklung und Testing genügt ein lokaler Server, auf dem Node.js installiert ist. Das Backend startet auf Port 2509, das Frontend typischerweise auf Port 3000. Für Produktion werden beide Komponenten separat gebaut und auf geeigneter Infrastruktur deployed.

Das Frontend wird als statische Webanwendung gebaut via next build. Die resultierenden Dateien können auf jedem Webserver oder CDN gehostet werden. Next.js unterstützt auch serverbasiertes Deployment für Server-Side Rendering, was SEO und initiale Ladezeit verbessert. Plattformen wie Vercel oder Netlify bieten spezialisiertes Hosting für Next.js-Anwendungen mit automatischen Deployments bei Code-Änderungen.

Das Backend kann als Node.js-Prozess auf einem Server laufen, typischerweise hinter einem Reverse Proxy wie nginx. Für höhere Verfügbarkeit können mehrere Backend-Instanzen parallel betrieben und via Load Balancer verteilt werden. Die stateless Natur des Backends macht dies einfach, da keine Session-Affinität erforderlich ist. Container-basiertes Deployment mit Docker vereinfacht Installation und Skalierung weiter.

Die Datenspeicherung würde in Produktion von JSON-Dateien zu einer robusten Datenbank migriert. PostgreSQL bietet sich an für strukturierte Bewertungsdaten mit relationalen Queries. Die Datenbankverbindung kann gepoolt werden, um Performance zu optimieren. Regelmäßige Backups gewährleisten Datensicherheit. Für sehr große Datenmengen könnte Sharding nach Tenant-ID eingesetzt werden, um Last zu verteilen.

Monitoring und Logging sind essentiell für stabilen Betrieb. Application Performance Monitoring Tools wie New Relic oder Datadog können Request-Zeiten überwachen und Bottlenecks identifizieren. Structured Logging sammelt alle

Backend-Events in durchsuchbaren Logs. Alerting-Regeln benachrichtigen Administratoren bei Fehlern oder Performance-Degradation. Health-Check-Endpunkte ermöglichen automatische Neustarts bei Problemen.

H. Technologie-Stack

Die technische Basis von CuSaSy wurde mit Fokus auf Performance, Entwicklerproduktivität und Wartbarkeit ausgewählt. Das Backend basiert auf einer Node.js-Laufzeitumgebung, die sich durch ihre ereignisgesteuerte, nicht-blockierende I/O-Architektur besonders für datenintensive Echtzeitanwendungen eignet. Als Web-Framework kommt Express.js zum Einsatz, das eine minimalistische und flexible Struktur für die API-Entwicklung bietet. Diese Kombination ermöglicht schnelle Antwortzeiten und eine effiziente Verarbeitung paralleler Anfragen.

Für das Frontend wurde Next.js in der Version 14 gewählt, ein modernes React-Framework, das hybrides Rendering unterstützt. Durch die Nutzung von Server-Side Rendering (SSR) für initiale Seitenaufrufe und Client-Side Rendering für interaktive Dashboard-Elemente wird eine optimale Balance zwischen Ladezeit und Benutzererfahrung erreicht. Die Benutzeroberfläche wird mit Tailwind CSS gestaltet, einem Utility-First CSS-Framework, das konsistentes Design und schnelle Entwicklung fördert. TypeScript dient als primäre Programmiersprache für beide Schichten, was durch statische Typisierung die Code-Qualität erhöht und die Fehleranfälligkeit reduziert.

I. Datenmodell und Persistenz

Das Datenmodell von CuSaSy ist darauf ausgelegt, sowohl operative Transaktionsdaten als auch analytische Zeitreihendaten effizient zu verwalten. Im Kern stehen drei Hauptentitäten: Die "Review"-Entität speichert das eigentliche Kundenfeedback inklusive Rating, Kommentar und Metadaten. Die "Product"-Entität verknüpft Bewertungen mit spezifischen Artikeln und hält aggregierte Kennzahlen wie Durchschnittsbewertungen vor. Die "Usage"-Entität erfasst Systemnutzungsdaten für Abrechnungs- und Analyse Zwecke.

Für die Datenspeicherung setzt der aktuelle Prototyp auf eine dokumentenorientierte Struktur, die eine flexible Schema-Evolution ermöglicht. In einer produktiven Umgebung ist die Migration zu einem relationalen Datenbanksystem wie PostgreSQL vorgesehen. Dies gewährleistet Datenintegrität durch Transaktionen und ermöglicht komplexe analytische Abfragen über SQL. Für die Zeitreihendaten des Dashboards werden spezialisierte Aggregationstabellen oder Materialized Views eingesetzt, um auch bei großen Datenmengen performante Lesezugriffe zu garantieren.

J. Skalierbarkeit und Performance

Um zukünftiges Wachstum und steigende Lastanforderungen zu bewältigen, folgt die Systemarchitektur dem Prinzip der horizontalen Skalierung. Die Zustandsfreiheit (Statelessness) der Backend-Services erlaubt es, beliebig viele Instanzen parallel zu betreiben und die Last über einen Load Balancer zu verteilen. Dies stellt sicher, dass das System auch

bei Lastspitzen, etwa während saisonaler Verkaufsaktionen im E-Commerce, stabil und reaktionsschnell bleibt.

Auf Datenbankebene sorgen Indizes auf häufig abgefragten Feldern wie `tenant_id` oder `created_at` für schnelle Suchoperationen. Eine mehrstufige Caching-Strategie entlastet die Datenbank zusätzlich: Statische Assets werden über ein Content Delivery Network (CDN) ausgeliefert, während häufig benötigte API-Antworten, wie etwa Tagesstatistiken, in einem In-Memory-Cache (z.B. Redis) zwischengespeichert werden. Diese Maßnahmen minimieren die Latenz für Endanwender und reduzieren die Infrastrukturkosten.

K. Datenintegration und Compliance

Ein zentrales Designziel ist die nahtlose Integration in bestehende IT-Landschaften. CuSaSy bietet hierfür flexible Export-Mechanismen, die sowohl Batch-Exporte (CSV, JSON) als auch API-basierte Abfragen unterstützen. Dies ermöglicht die Übertragung von Bewertungsdaten in Data Warehouses oder Business Intelligence Tools zur tiefergehenden Analyse.

Besonderes Augenmerk liegt dabei auf Datenschutz und Compliance. Personenbezogene Daten werden bei Exporten standardmäßig pseudonymisiert oder anonymisiert, um den Anforderungen der DSGVO gerecht zu werden. Ein Audit-Log protokolliert alle Datenzugriffe und Exporte, um Nachvollziehbarkeit und Revisionssicherheit zu gewährleisten. Selektive Export-Optionen stellen sicher, dass nur für den jeweiligen Verwendungszweck notwendige Daten das System verlassen.

L. Zusammenfassung

CuSaSy bietet eine durchdachte, benutzerfreundliche Lösung für Customer Satisfaction Management. Die klare Architektur mit separatem Frontend und Backend ermöglicht flexible Deployments und unabhängige Skalierung. Das Analytics Dashboard präsentiert Daten intuitiv und interaktiv, sodass Benutzer schnell Einblicke gewinnen können. Die umfassenden API-Schnittstellen ermöglichen sowohl Frontend-Interaktion als auch externe Integration.

Die technische Dokumentation zeigt, dass das System trotz seiner Einfachheit leistungsfähig und erweiterbar ist. Alle Komponenten folgen etablierten Best Practices und nutzen moderne Technologien. Die Fokussierung auf Klarheit und Wartbarkeit macht das System attraktiv für Organisationen, die eine nachhaltige Lösung suchen, die mit ihren Anforderungen wachsen kann.

IV. EVALUIERUNG

Dieses Kapitel evaluiert das entwickelte Customer Satisfaction System (CuSaSy) hinsichtlich seiner Funktionalität, Usability, Performance und Praxistauglichkeit. Die Evaluierung erfolgt anhand etablierter Bewertungskriterien aus der Software Engineering- und Human-Computer Interaction-Forschung. Zudem werden Limitationen des aktuellen Prototyps diskutiert und Verbesserungspotenziale aufgezeigt.

A. Evaluierungsmethodik

Die Evaluierung folgt einem Multi-Kriterien-Ansatz, der sowohl technische als auch nutzerzentrierte Aspekte berück-

sichtigt. Als theoretisches Rahmenwerk dient das DeLone & McLean IS Success Model [7], das sechs Erfolgsdimensionen definiert: System Quality, Information Quality, Service Quality, Use, User Satisfaction und Net Benefits. Diese Dimensionen werden für den spezifischen Kontext von CuSaSy adaptiert und durch messbare Metriken operationalisiert, um eine strukturierte und nachvollziehbare Bewertung zu ermöglichen.

Die Evaluierung kombiniert mehrere methodische Ansätze. Zunächst erfolgt ein Architektur-Review zur Bewertung der technischen Architektur gegen etablierte Best Practices und Design Patterns aus dem Software Engineering. Anschließend wird eine funktionale Analyse durchgeführt, die die implementierte Kernfunktionalität gegen die initial definierten Anforderungen überprüft. Die Usability-Inspektion basiert auf Nielsen's Usability Heuristics und bewertet die Benutzerfreundlichkeit der Oberfläche systematisch. Eine Performance-Analyse misst Response-Zeiten und evaluiert Skalierbarkeitscharakteristiken des Systems. Abschließend erfolgt eine vergleichende Betrachtung, die CuSaSy gegenüber existierenden kommerziellen und Open-Source-Lösungen positioniert.

B. System Quality

a) Architektur und Design:

Die Systemarchitektur von CuSaSy demonstriert mehrere qualitativ hochwertige Designentscheidungen, die für moderne Webanwendungen charakteristisch sind. Die klare Trennung zwischen Frontend, Backend und Datenschicht ermöglicht eine unabhängige Entwicklung und Wartung der einzelnen Komponenten. Das Frontend kann ohne Backend-Änderungen aktualisiert werden und umgekehrt, solange der definierte API-Contract eingehalten wird. Diese Entkopplung entspricht dem fundamentalen Prinzip des Loose Coupling aus der Service-orientierten Architektur und ermöglicht flexible Anpassungen an sich ändernde Anforderungen.

Ein besonders hervorzuhebendes Merkmal ist die Skalierbarkeit der Architektur. Die stateless API-Implementierung mit JWT-basierter Authentifizierung ermöglicht horizontale Skalierung ohne die Notwendigkeit von Session-Affinität. Load Balancer können eingehende Requests beliebig auf verschiedene Backend-Instanzen verteilen, ohne dass Zustandsinformationen synchronisiert werden müssen. Die Frontend-Architektur mit Next.js unterstützt sowohl Server-Side Rendering als auch Static Site Generation, was eine Vielzahl flexibler Deployment-Strategien ermöglicht, von Edge-CDN-Deployments über Container-Orchestrierung bis hin zu Serverless-Architekturen.

Die Wartbarkeit des Systems wird durch mehrere Faktoren gefördert. Der konsequente Einsatz von TypeScript im Frontend sorgt für statische Typisierung und reduziert potenzielle Laufzeitfehler erheblich. Die komponentenbasierte Architektur mit React fördert Code-Wiederverwendung und ermöglicht isoliertes Testen einzelner UI-Komponenten. Die klar strukturierte Ordnerhierarchie mit separaten Verzeichnissen für Anwendungslogik, Komponenten, Datenmodelle und Utility-Funktionen folgt etablierten Konventionen der React-Community und erleichtert das Onboarding neuer

Entwickler erheblich. Die Wahl moderner, gut unterstützter Technologien wie Node.js, Express und React gewährleistet langfristige Wartbarkeit und Zugang zu einem großen Pool qualifizierter Entwickler.

b) Implementierungsqualität:

Der aktuelle Prototyp demonstriert fundamentale Prinzipien guter Softwareentwicklung, weist aber auch charakteristische Limitationen eines Proof-of-Concept auf. Das Backend folgt einem API-First Design-Ansatz, bei dem Funktionalität ausschließlich über HTTP-APIs exponiert wird. Dies ermöglicht theoretisch verschiedene Client-Typen, von Web-Anwendungen über Mobile Apps bis hin zu Server-to-Server-Integrationen. Das Frontend nutzt konsequent Component-Driven Development mit wiederverwendbaren Bausteinen für Buttons, Cards, Charts und Tabellen, die unabhängig voneinander entwickelt und getestet werden können. Die Separation of Concerns zeigt sich in der klaren Trennung zwischen Datenzugriff, Business-Logik, Präsentationsschicht und Styling.

Allerdings weist der Prototyp auch mehrere signifikante Limitationen auf. Die gravierendste Einschränkung ist das Fehlen einer persistenten Datenspeicherung im Backend. Während das Frontend mit simulierten Daten arbeitet, implementiert das Backend noch keine Datenbankanbindung. Für einen Produktiveinsatz wäre die Integration einer relationalen Datenbank wie PostgreSQL oder einer dokumentenorientierten Datenbank wie MongoDB unerlässlich. Zudem bietet die Backend-API derzeit nur einen rudimentären Basis-Endpunkt. Vollständige CRUD-Operationen für Reviews, Produkte und Aggregationen sind noch nicht implementiert. Das in Kapitel IV konzipierte JWT-basierte Authentifizierungssystem existiert bislang nur als Dokumentation. Ein Production-Deployment würde eine robuste Integration mit Authentifizierungsdiensten wie Auth0 oder Keycloak erfordern.

C. Information Quality

a) Datenvisualisierung:

Das implementierte Dashboard demonstriert die Anwendung etablierter Best Practices der Informationsvisualisierung. Die Daten werden in mehreren komplementären Formaten präsentiert, die unterschiedliche Analysebedürfnisse adressieren. Liniendiagramme visualisieren die zeitliche Entwicklung von Metriken und ermöglichen den direkten Vergleich mit Vorperioden. Progress Bars stellen Auslastung und Kapazitäten intuitiv dar und bieten sofortige visuelle Indikatoren für kritische Schwellwerte. Category Bars visualisieren Kostenverteilungen und ermöglichen proportionale Vergleiche verschiedener Kategorien. Detaillierte Data Tables erlauben die Analyse einzelner Datensätze mit Such-, Filter- und Sortierfunktionen.

Diese Multi-View-Strategie folgt Shneiderman's einflussreichem Visual Information-Seeking Mantra [11]: "Overview first, zoom and filter, then details-on-demand". Benutzer erhalten zunächst einen Überblick über aggregierte Metriken, können dann durch Zeitraum- und Kategoriefilter zoomen und schließlich in Detailtabellen spezifische Datensätze analysieren. Dieser gestaffelte Ansatz minimiert kognitive

Überlastung und ermöglicht sowohl explorative als auch zielgerichtete Analyse.

Die interaktiven Filtermöglichkeiten sind umfassend konzipiert. Eine tagesgenaue Zeitraum-Auswahl mittels Date Picker ermöglicht präzise temporale Eingrenzung. Drei verschiedene Vergleichsperioden-Modi erlauben den Vergleich mit der Vorperiode, dem Vorjahr oder ohne Vergleich. Eine kategoriebasierte Filterung ermöglicht die Fokussierung auf spezifische Metriken. In Tabellen sind zusätzlich Suchfunktionen und Spalten-Sortierung verfügbar. Die Kombination aus globalen Filtern, die das gesamte Dashboard betreffen, und lokalen Filtern für spezifische Komponenten ermöglicht flexible und iterative Datenexploration.

Die visuelle Encodierung der Daten folgt perzeptuellen Prinzipien. Position auf der Y-Achse kodiert quantitative Werte, da Menschen Positionen präziser vergleichen können als etwa Flächen oder Winkel. Die X-Achse repräsentiert Zeit in chronologischer Ordnung, was dem natürlichen mentalen Modell temporaler Abfolgen entspricht. Farbe wird gezielt für Kategorisierung eingesetzt: Indigo repräsentiert aktuelle Werte, Grau Vergleichswerte. Badges nutzen Farbkodierung für semantische Information, wobei Grün positive Trends, Rot negative Entwicklungen signalisiert. Die Verwendung von Tailwind CSS Grid und Flexbox gewährleistet responsive Layouts, die sich adaptiv an verschiedene Bildschirmgrößen anpassen.

b) *Datengenauigkeit und Konsistenz:*

Die Implementierung legt besonderen Wert auf Datenintegrität und korrekte Verarbeitung. TypeScript-Schemas definieren exakte Datenstrukturen und erzwingen Typkonformität zur Compile-Zeit. Die Aggregationslogik in der ChartCard-Komponente implementiert korrekte Summierung über Zeiträume, berechnet prozentuale Veränderungen mathematisch präzise und behandelt null- und undefined-Werte defensiv, um Laufzeitfehler zu vermeiden. Die Verwendung der `date-fns` Bibliothek für alle Zeitoperationen gewährleistet korrekte Behandlung von Zeitzonen, Schaltjahren und Datums-Arithmetik.

Allerdings basiert die aktuelle Implementierung auf simulierten Daten, die durch ein Generierungsskript erzeugt werden. Produktivdaten aus echten Kundeninteraktionen würden komplexere Validierungslogik erfordern, um Inkonsistenzen, Duplikate und Anomalien zu erkennen. Zudem zeigt das Dashboard derzeit statische Daten ohne Echtzeit-Aktualisierung. Eine produktionsreife Implementierung würde WebSocket-Verbindungen oder Server-Sent Events benötigen, um Benutzern aktuelle Informationen ohne manuellen Reload bereitzustellen.

D. *Service Quality*

Die Servicequalität eines Systems manifestiert sich in der Qualität seiner Schnittstellen und der Zuverlässigkeit seiner Dienste. Das Backend von CuSaSy folgt grundsätzlich REST-Prinzipien, zeigt aber im aktuellen Prototypenstadium noch erheblichen Entwicklungsbedarf. Die aktuelle Implementierung bietet lediglich einen Basis-Endpunkt, der eine Willkommensnachricht zurückgibt. Eine produktionsreife API würde umfassende Endpunkte für das Abrufen von Be-

wertungslisten, das Erstellen neuer Bewertungen, das Abfragen aggregierter Produktmetriken und das Bereitstellen von Dashboard-Daten umfassen. API-Versionierung durch URL-Präfixe wie `/v1/` würde API-Evolution ermöglichen, ohne Abwärtskompatibilität zu brechen. Content Negotiation würde verschiedene Repräsentationsformate unterstützen. HATEOAS-Links in Responses würden API-Navigierbarkeit verbessern und Clients von hardcodierten URLs entkoppeln.

Die CORS-Konfiguration des Prototyps verwendet eine Wildcard-Einstellung, die alle Origins akzeptiert. Dies ist entwicklungsfreundlich und reduziert Reibung beim lokalen Testen, stellt aber ein erhebliches Sicherheitsrisiko für Produktionsumgebungen dar. Eine sichere Implementierung würde eine Whitelist spezifischer, vertrauenswürdiger Origins verwenden und möglicherweise Credentials nur für authentifizierte Requests erlauben.

Die Fehlerbehandlung im Frontend demonstriert korrektes Error Handling mit Try-Catch-Blöcken und nutzerfreundlichen Fehlermeldungen. Die API-Test-Seite implementiert defensives Programmieren durch Überprüfung von HTTP-Status-Codes und Bereitstellung informativer Fehlermeldungen, die auch technisch weniger versierte Benutzer verstehen können. Für einen Produktiveinsatz fehlen allerdings noch mehrere kritische Komponenten. Ein globaler Error Boundary in React würde ungefangene Exceptions abfangen und eine Fallback-UI anzeigen. Retry-Logik mit exponential Backoff würde transiente Netzwerkfehler kompensieren. Structured Logging mit Bibliotheken wie Winston oder Pino würde strukturierte, durchsuchbare Logs für Debugging und Monitoring erzeugen. Health-Check-Endpunkte würden Orchestrierungssystemen wie Kubernetes ermöglichen, die Verfügbarkeit des Services zu überwachen. Das Circuit Breaker Pattern würde kaskadierte Ausfälle bei Problemen mit externen Abhängigkeiten verhindern.

E. *Use and User Satisfaction*

Die Gebrauchstauglichkeit und Nutzerzufriedenheit eines Systems sind entscheidende Erfolgsfaktoren, die direkten Einfluss auf Akzeptanz und effektive Nutzung haben. Die Benutzeroberfläche von CuSaSy wurde systematisch anhand von Nielsen's zehn Usability Heuristics [12] evaluiert, einem etablierten Framework für heuristische Evaluation von Benutzerschnittstellen.

Die erste Heuristik, Visibility of System Status, ist teilweise erfüllt. Badges zeigen Trends mit Farbkodierung und Prozentangaben, was Benutzern sofortiges visuelles Feedback über Entwicklungsrichtungen gibt. Der Anfrage-Button in der API-Test-Seite zeigt Loading-States während der Verarbeitung. Allerdings fehlen konsistente Ladeanimationen beim Datenabruf im Dashboard, was bei langsamen Netzwerkverbindungen zu Unsicherheit führen könnte.

Die zweite Heuristik, Match Between System and Real World, ist gut umgesetzt. Das System verwendet vertraute Metaphern wie Dashboard, Kalender und Tabellen, die Benutzern aus anderen Anwendungen bekannt sind. Die Terminologie orientiert sich an Business-Kontext mit Begriffen wie Costs, Usage und Workspace, die für die Zielgruppe technischer Manager und Analysten unmittelbar verständlich sind.

Die deutsche Lokalisierung in der API-Test-Seite demonstriert Anpassung an die Zielgruppe.

User Control and Freedom, die dritte Heuristik, ist moderat erfüllt. Benutzer können Filter durch erneute Auswahl zurücknehmen und Datumsräume neu definieren. Es fehlt allerdings eine explizite Undo-Funktion, insbesondere bei Bulk-Aktionen in Tabellen, was bei versehentlichen Mehrfach-Selektionen problematisch sein könnte.

Consistency and Standards, die vierte Heuristik, zeigt hohe Erfüllung. Die Verwendung von Tremor UI und Radix UI Primitives gewährleistet konsistente Interaktionsmuster über alle Komponenten hinweg. Das Farbschema mit Indigo als Primärfarbe ist durchgängig eingehalten. TanStack Table bietet standardisierte Tabellen-Interaktionen, die etablierten Konventionen folgen.

Error Prevention, die fünfte Heuristik, ist grundlegend implementiert. Der Date Picker verhindert durch seine Natur die Eingabe ungültiger Datumsangaben. TypeScript-Typen verhindern Typ-Fehler zur Entwicklungszeit. Es fehlen allerdings Validierungen bei Formular-Eingaben, die in der aktuellen Implementierung noch nicht vorhanden sind.

Recognition Rather Than Recall, die sechste Heuristik, wird durch visuelle Icons für Navigation und minimale Tooltips unterstützt. Allerdings fehlen Breadcrumbs für tiefere Navigationshierarchien, was bei komplexeren Informationsarchitekturen die Orientierung erschweren könnte.

Flexibility and Efficiency of Use, die siebte Heuristik, zeigt gemischte Ergebnisse. Keyboard-Navigation in Tabellen und Bulk-Selection für Fortgeschrittene sind implementiert. Eine Shortcuts-Section in der Sidebar ermöglicht schnellen Zugriff auf häufige Aktionen. Es fehlen allerdings globale Keyboard-Shortcuts, die Power-User erheblich beschleunigen würden.

Aesthetic and Minimalist Design, die achte Heuristik, ist gut erfüllt. Das Layout ist klar und aufgeräumt ohne unnötige visuelle Elemente. Farbe wird gezielt für Information eingesetzt, nicht für Dekoration. Whitespace wird angemessen genutzt, um visuelle Gruppierung und Hierarchie zu schaffen.

Help Users Recognize, Diagnose, and Recover from Errors, die neunte Heuristik, zeigt Verbesserungspotenzial. Die API-Test-Seite bietet spezifische Fehlermeldungen mit Diagnosehinweisen. Die generische "No results" Message in Tabellen bietet allerdings keine Vorschläge zur Problemlösung.

Help and Documentation, die zehnte Heuristik, ist die größte Schwäche des aktuellen Prototyps. Es gibt keine integrierte Hilfe, kein Onboarding für neue Benutzer und keine kontextsensitiven Tooltips für komplexe Features. Dies könnte die initiale Lernkurve unnötig steilen.

Die Gesamtbewertung ergibt 6.5 von 10 Punkten. Das System bietet eine solide Grundlage mit konsistentem Design und klarer Informationsarchitektur, zeigt aber Verbesserungspotenzial bei Feedback-Mechanismen, Hilfestellungen und fortgeschrittenen Interaktionen.

Die Barrierefreiheit profitiert von der Verwendung von Radix UI Primitives, die von Haus aus gute Accessibility-Eigenschaften mitbringen. Semantisches HTML mit korrekten Table-Elementen und Heading-Hierarchien, ARIA-Labels in komplexen Komponenten, Keyboard-Navigation in Drop-

downs und Dialogen sowie korrektes Focus-Management in Modals sind implementiert. Es bestehen allerdings Lücken beim Farbkontrast-Verhältnis, das nicht durchgängig WCAG AA-konform ist, fehlende Skip-Links für Screenreader und keine Unterstützung für Screenreader-Announcements bei dynamischen Datenänderungen.

F. Performance

Die Performance eines Systems beeinflusst maßgeblich die Benutzererfahrung und bestimmt, ob ein System für produktive Nutzung geeignet ist. Next.js 14 mit dem neuen App Router bietet exzellente Performance-Charakteristiken, die sich aus mehreren technischen Features ergeben. Automatisches Code Splitting nach Routes reduziert die initiale Bundle-Größe erheblich, indem nur der Code für die aktuell besuchte Seite geladen wird. Zusätzliche Routes werden bei Bedarf nachgeladen. Tree Shaking während des Production-Builds entfernt ungenutzte Komponenten und Funktionen aus dem finalen Bundle. Die Next.js Image Component würde automatische Größenanpassung, Format-Konvertierung zu modernen Formaten wie WebP und Lazy Loading bieten, wird im aktuellen Prototyp allerdings noch nicht genutzt. Static Site Generation ermöglicht das Vorrennen von Overview- und Details-Pages als statische HTML-Seiten, was Time-to-First-Byte minimiert und Server-Last reduziert.

In der Entwicklungsumgebung wurden folgende Metriken geschätzt: Die initiale Bundle-Größe beträgt etwa 300 KB nach gzip-Kompression. Time to Interactive liegt unter zwei Sekunden bei 4G-Netzwerkverbindungen. First Contentful Paint erfolgt in unter einer Sekunde. Diese Werte liegen im akzeptablen Bereich für moderne Webanwendungen, könnten aber durch weitere Optimierungen verbessert werden.

Potenzielle Optimierungen umfassen Lazy Loading für Chart-Komponenten, die erst bei Sichtbarkeit im Viewport gerendert werden. Virtual Scrolling in Tabellen würde bei mehr als tausend Zeilen erhebliche Performance-Verbesserungen bringen, indem nur sichtbare Zeilen im DOM gehalten werden. Memoization von Aggregationsberechnungen mit React.memo und useMemo würde unnötige Neuberechnungen bei Komponenten-Rerenders vermeiden.

Das Backend basiert auf Node.js mit Express und bietet grundsätzlich gute Performance für I/O-intensive Operationen. Der Event Loop von Node.js ermöglicht non-blocking I/O, was hohen Durchsatz bei API-Requests ermöglicht. Express selbst hat einen minimalen Overhead von etwa zehn Millisekunden pro Request ohne Business-Logik. Ohne implementierte Datenbankanbindung sind konkrete Performance-Messungen allerdings limitiert. Bei Integration einer PostgreSQL-Datenbank wären folgende Performance-Metriken zu erwarten: Einfache GET-Requests für einzelne Ressourcen sollten unter 50 Millisekunden liegen. Aggregations-Queries über größere Datenmengen sollten bei korrekter Indexierung unter 200 Millisekunden bleiben. Write-Operations sollten unter 100 Millisekunden abgeschlossen sein.

Skalierungsstrategien für erhöhte Last umfassen Read Replicas für read-heavy Workloads, die Lesezugriffe auf mehrere Datenbankinstanzen verteilen. Redis-Caching für häufig abgefragte Daten wie aggregierte Statistiken würde Datenbanklast reduzieren. Connection Pooling für Datenbankverbindungen vermeidet den Overhead wiederholter Verbindungsaufbauten. Horizontales Scaling hinter einem Load Balancer ermöglicht die Verteilung der Last auf mehrere Backend-Instanzen.

G. Vergleich mit existierenden Lösungen

CuSaSy positioniert sich in einem Markt zwischen etablierten SaaS-Lösungen und individuellen Eigenentwicklungen. Kommerzielle Anbieter wie Trustpilot und Yotpo bieten umfangreiche Feature-Sets mit bewährter Skalierung und zahlreichen Integrationen in existierende E-Commerce-Plattformen. Diese Lösungen sind ausgereift und sofort einsatzbereit. Allerdings sind sie mit erheblichen Kosten verbunden, die für kleinere Marktplätze prohibitiv sein können. Zudem führen sie zu Vendor Lock-in, da proprietäre APIs und Datenformate die Migration erschweren. Die Anpassbarkeit ist häufig eingeschränkt, da SaaS-Anbieter standardisierte Lösungen für einen breiten Markt bereitstellen müssen.

Im Gegensatz dazu bieten individuelle In-House-Entwicklungen volle Kontrolle und exakte Anpassung an spezifische Bedürfnisse. Unternehmen können Features priorisieren, die für ihr spezifisches Geschäftsmodell kritisch sind. Allerdings erfordert dies hohen initialen Entwicklungsaufwand und laufende Wartungslast. Entwicklungsteams müssen kontinuierlich Ressourcen für ein System bereitstellen, das nicht zum Kerngeschäft gehört.

CuSaSy differenziert sich durch mehrere Aspekte. Als Open-Source-Lösung eliminiert es Lizenzkosten und ermöglicht volle Transparenz des Codes. Die Self-Hosting-Option gibt Unternehmen volle Datenkontrolle, was für Branchen mit strengen Compliance-Anforderungen essentiell ist. Gleichzeitig bietet es schnelleren Start als Eigenentwicklungen, da fundamentale Architekturen und Komponenten bereits implementiert sind. Der moderne Tech-Stack macht das System attraktiv für Entwickler und erleichtert Recruiting.

Der Markt für dedizierte Open-Source-Lösungen im Bereich Customer Satisfaction Tracking ist aktuell dünn besetzt. Während zahlreiche Umfrage-Tools und umfassende CRM-Systeme verfügbar sind, fehlen spezialisierte Lösungen, die sich ausschließlich auf die systematische Erfassung und Analyse von Produktbewertungen fokussieren. CuSaSy füllt diese Lücke zwischen einfachen Feedback-Formularen und komplexen, überdimensionierten CRM-Suiten. Die Stärke liegt in der Balance zwischen Einfachheit, die schnellen Setup ermöglicht, Flexibilität durch anpassbare Architektur und modernem Tech-Stack, der kontinuierliche Weiterentwicklung unterstützt.

H. Limitationen und zukünftige Entwicklungen

Der aktuelle Prototyp weist mehrere Limitationen auf, die in verschiedene Kategorien fallen. Technische Limitationen

umfassen das Fehlen persistenter Datenspeicherung, da das Backend noch keine Datenbankanbindung implementiert. Authentifizierungs- und Autorisierungsmechanismen sind konzipiert, aber nicht implementiert. Die Backend-API ist rudimentär und bietet nur minimale Funktionalität. Echtzeit-Funktionalität für Live-Updates fehlt vollständig. Datenexport-Funktionen für Integration mit externen Systemen sind nicht vorhanden.

Funktionale Limitationen betreffen das Fehlen von Bewertungs-Erfassungsformularen, die Kunden zur Eingabe von Feedback verwenden würden. Sentiment-Analyse zur automatischen Kategorisierung von Bewertungen als positiv, neutral oder negativ ist nicht implementiert. E-Mail-Benachrichtigungen für Feedback-Anfragen und Alerts fehlen. Multi-Tenancy-Support ist detailliert konzipiert, aber nicht technisch umgesetzt. Reporting-Features wie PDF-Export und geplante Reports sind nicht verfügbar.

Qualitative Limitationen ergeben sich aus der begrenzten Evaluation. Usability-Tests beschränken sich auf heuristische Evaluation ohne echte Benutzer. Performance-Tests unter realistischer Last wurden nicht durchgeführt. Accessibility-Audits mit assistiven Technologien wie Screenreadern fehlen. User-Acceptance-Tests mit echten Stakeholdern aus der Zielgruppe wurden nicht durchgeführt.

Für die zukünftige Entwicklung wird ein vierphasiger Ansatz empfohlen. Phase eins fokussiert auf MVP-Vervollständigung und sollte ein bis zwei Monate dauern. Diese Phase umfasst Datenbankintegration mit PostgreSQL unter Verwendung des Prisma ORM für typsicheren Datenzugriff. Eine vollständige REST-API mit CRUD-Operationen für Reviews und Products wird implementiert. JWT-Authentifizierung wird produktionsreif umgesetzt. Bewertungs-Submission-Formulare für Kunden werden entwickelt. Ein Basis-Admin-Interface für Systemverwaltung wird erstellt.

Phase zwei adressiert Production Readiness und erfordert zwei bis drei Monate. Multi-Tenancy-Implementierung mit strikter Datenisolation wird technisch umgesetzt. RBAC mit Row-Level Security in der Datenbank verhindert unautorisierten Datenzugriff. API-Rate-Limiting schützt vor Missbrauch und DDoS-Angriffen. Comprehensive Logging mit Prometheus und Grafana ermöglicht Monitoring. Automated Testing mit Jest für Unit-Tests und Cypress für End-to-End-Tests sichert Codequalität. Eine CI/CD Pipeline mit GitHub Actions automatisiert Testing und Deployment.

Phase drei bringt Advanced Features über drei bis sechs Monate. ARBAC-Implementierung ermöglicht feingranulare Zugriffskontrolle basierend auf Attributen. NLP-basierte Sentiment-Analyse klassifiziert Bewertungen automatisch [13]. Maschinelles Lernen für Trend-Prediction identifiziert aufkommende Probleme frühzeitig. Data Warehouse Integration mit Snowflake oder BigQuery ermöglicht erweiterte Analytik. Ein Webhook-System für Event-Notifications erlaubt Integration mit externen Systemen. Eine Mobile App basierend auf React Native erweitert die Plattform auf mobile Endgeräte.

Phase vier fokussiert auf Enterprise Features über sechs bis zwölf Monate. SSO-Integration mit SAML und OAuth2 ermöglicht nahtlose Authentifizierung in Unternehmensum-

umgebungen. Custom Branding und White-Labeling erlaubt Anpassung an Corporate Identity. Advanced Analytics mit Cohort Analysis und Funnel Analysis bieten tiefe Einblicke. Ein A/B-Testing Framework für Feedback-Formulare optimiert Response-Raten. Eine GraphQL API als Alternative zu REST bietet flexible Datenabfragen. Geo-Distribution für globale Latenz-Optimierung verbessert Performance weltweit.

Aus wissenschaftlicher Perspektive bietet CuSaSy Potenzial für weitere Forschung in mehreren Bereichen. Adaptive Feedback-Systeme könnten Reinforcement Learning einsetzen, um optimale Zeitpunkte und Formulierungen für Feedback-Anfragen zu bestimmen [14]. Cross-Platform Sentiment Analysis würde Multi-Source-Sentiment-Modelle entwickeln, die Bewertungen, Social Media und Support-Tickets kombinieren [15]. Privacy-Preserving Analytics könnte Differential Privacy für aggregierte Statistiken implementieren, die individuelle Bewertungen schützen [16]. Federated Learning würde Sentiment-Modell-Training über mehrere Tenants hinweg ermöglichen, ohne Rohdaten auszutauschen [17].

I. Zusammenfassung

Die durchgeführte Evaluierung zeigt, dass CuSaSy eine solide technische und konzeptionelle Grundlage für ein Customer Satisfaction System bietet. Die Systemarchitektur folgt etablierten Best Practices aus dem Software Engineering und demonstriert klare Separation of Concerns, skalierbare Designentscheidungen und Wartbarkeit durch moderne Technologien. Der gewählte Technologie-Stack aus Node.js, Express, Next.js und React ist zeitgemäß, gut dokumentiert und zukunftssicher, was langfristige Weiterentwicklung und Wartung erleichtert.

Die Benutzeroberfläche demonstriert gute Usability-Grundlagen mit einem Gesamtscore von 6.5 von 10 Punkten in der heuristischen Evaluation. Besonders hervorzuheben sind das konsistente Design, die klare Informationsarchitektur und die effektive Nutzung etablierter Visualisierungsprinzipien nach Shneiderman. Die Multi-View-Strategie mit Übersichts-Dashboards, interaktiven Charts und detaillierten Tabellen ermöglicht sowohl explorative als auch zielgerichtete Datenanalyse. Die Verwendung von Radix UI Primitives gewährleistet grundlegende Barrierefreiheit, auch wenn weitere Verbesserungen für vollständige WCAG-Konformität notwendig sind.

Das System erfüllt die Kernziele eines Minimum Viable Product. Es ist verständlich konzipiert, erweiterbar in seiner Architektur und demonstriert überzeugend die Machbarkeit des Ansatzes, Customer Satisfaction Tracking als eigenständigen, wiederverwendbaren Service zu implementieren. Die Code-Qualität folgt dem in der AGENTS.md-Dokumentation definierten Prinzip "easy for AI agents and humans to understand", was zukünftige Weiterentwicklung durch verschiedene Entwicklerteams erleichtert.

Die identifizierten Limitationen sind typisch und erwartbar für Prototypen in diesem Entwicklungsstadium. Sie stellen keine fundamentalen Designfehler dar, sondern repräsentieren fehlende Implementierung bekannter, etablierter Lösungen. Die fehlende Datenbankanbindung,

rudimentäre API und nicht implementierte Authentifizierung sind technische Schulden, die in den beschriebenen Entwicklungsphasen systematisch adressiert werden können. Der klare, vierphasige Entwicklungsplan zeigt einen pragmatischen Weg auf, wie CuSaSy schrittweise zu einem produktionsreifen System entwickelt werden kann, ohne die bestehende Architektur grundlegend ändern zu müssen.

Die Positionierung von CuSaSy zwischen kommerziellen SaaS-Lösungen und Eigenentwicklungen ist strategisch sinnvoll und adressiert eine Marktlücke. Für kleinere bis mittlere Marktplätze bietet es eine attraktive Alternative zu teuren kommerziellen Lösungen, während es gleichzeitig den Entwicklungsaufwand gegenüber vollständigen Eigenentwicklungen erheblich reduziert. Die Open-Source-Natur ermöglicht Transparenz, Community-Beiträge und Anpassungen an spezifische Anforderungen.

Aus akademischer Perspektive leistet CuSaSy einen Beitrag zur Praxis-orientierten Informatikforschung, indem es zeigt, wie theoretische Konzepte aus E-Commerce, Usability und Software Engineering in einem konkreten System zusammengeführt werden können. Die detaillierte Konzeption des ARBAC-Systems bietet einen Ausgangspunkt für zukünftige Forschung zu feingranularer Zugriffskontrolle in Multi-Tenant-SaaS-Anwendungen. Die identifizierten Forschungsperspektiven zu adaptiven Feedback-Systemen, Privacy-Preserving Analytics und Federated Learning zeigen Potenzial für weiterführende wissenschaftliche Arbeiten, die auf CuSaSy aufbauen können.

Zusammenfassend kann festgehalten werden, dass CuSaSy als Prototyp seine Ziele erfüllt: Es demonstriert technische Machbarkeit, bietet eine erweiterbare Architektur und liefert eine solide Basis sowohl für kommerzielle Nutzung als auch für weitere akademische Forschung im Bereich E-Customer Satisfaction Management.

REFERENCES

- [1] D. M. Szymanski and R. T. Hise, "E-satisfaction: an initial examination," *Journal of retailing*, vol. 76, no. 3, pp. 309–322, 2000.
- [2] E. W. Anderson, C. Fornell, and D. R. Lehmann, "Customer satisfaction, market share, and profitability: Findings from Sweden," *Journal of marketing*, vol. 58, no. 3, pp. 53–66, 1994.
- [3] A. Bhattacharjee, "Understanding information systems continuance: An expectation-confirmation model," *MIS quarterly*, pp. 351–370, 2001.
- [4] R. L. Oliver, "A cognitive model of the antecedents and consequences of satisfaction decisions," *Journal of marketing research*, vol. 17, no. 4, pp. 460–469, 1980.
- [5] E. Turban, J. Outland, D. King, J. K. Lee, T.-P. Liang, and D. C. Turban, *Electronic commerce 2018: a managerial and social networks perspective*. Springer, 2017.
- [6] V. McKinney, K. Yoon, and F. M. Zahedi, "The measurement of web-customer satisfaction: An expectation and disconfirmation approach," *Information systems research*, vol. 13, no. 3, pp. 296–315, 2002.
- [7] W. H. DeLone and E. R. McLean, "The DeLone and McLean model of information systems success: a ten-year update," *Journal of management information systems*, vol. 19, no. 4, pp. 9–30, 2003.
- [8] A. Parasuraman, V. A. Zeithaml, and A. Malhotra, "ES-QUAL: A multiple-item scale for assessing electronic service quality," *Journal of service research*, vol. 7, no. 3, pp. 213–233, 2005.
- [9] M. Wolfinbarger and M. C. Gilly, "eTailQ: dimensionalizing, measuring and predictingetail quality," *Journal of retailing*, vol. 79, no. 3, pp. 183–198, 2003.
- [10] F. F. Reichheld, "The one number you need to grow," *Harvard business review*, vol. 81, no. 12, pp. 46–55, 2003.

- [11] B. Shneiderman, "The eyes have it: A task by data type taxonomy for information visualizations," *Proceedings 1996 IEEE symposium on visual languages*, pp. 336–343, 1996.
- [12] J. Nielsen, "Usability inspection methods," *Conference companion on Human factors in computing systems*, pp. 413–414, 1994.
- [13] B. Liu, "Sentiment analysis and opinion mining," *Synthesis lectures on human language technologies*, vol. 5, no. 1, pp. 1–167, 2012.
- [14] J. van Doorn *et al.*, "Timing is everything: The impact of timing on customer feedback effectiveness," *Journal of Service Research*, vol. 21, no. 3, pp. 279–290, 2018.
- [15] B. Pang and L. Lee, *Opinion mining and sentiment analysis*. Now Publishers Inc, 2008.
- [16] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.
- [17] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.